

Assignment 2

Author: Lan Stare **Date:** 11. 9. 2025

Question 1

After initialization, only 2 is in the queue. In the first loop we move 2 from the queue to the explored array and add 3 and 4 to the queue. We also remember that 2 as the predecessor of 3 and 4. In the next loop we then move 3 from the queue to the explored array, so the queue only has 4. The predecessor of 3 is 2. In the next loop we move 4 from the queue to the explored and add 5 to the queue. We also remember the predecessors of 5 are 4 and 2. In the next loop we move 5 from the queue to the explored and put 1 in the queue, remembering its predecessors. In the enxt step we move 1 from the queue to explored and end the algorithm, since the queue is now empty. We have not discovered the number 0.

Question 2

```
In [11]: def remove_adjacent_duplicates(s):
    stack = []
    for letter in s:
        if stack != [] and letter == stack[-1]:
            stack.pop()
        else:
            stack.append(letter)
    return ''.join(stack)

print(remove_adjacent_duplicates('aacacbbcab'))

cb
```

Question 3

```
In [71]: from collections import deque

example_graph2 = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['A', 'E'],
    'D': ['A'],
    'E': ['A']}

def bfs(graph, vertex):
    C = []

    # Initialize
    explored = {u: "UNDISCOVERED" for u in graph}
    predecessor = {u: None for u in graph}

    explored[vertex] = "DISCOVERED" #setting the given vertex as explored
    predecessor[vertex] = None

    Q = deque()
    Q.append(vertex) #adding the given vertex to the queue

    # Main Loop
    while Q: #Creating a loop for eash vertex if queue is not empty
        u = Q.popleft() #taking the first element of the queue
        for w in graph[u]:
            if explored[w] == "UNDISCOVERED":
                explored[w] = "DISCOVERED"
                predecessor[w] = u
                Q.append(w)
            elif w == vertex: #If vertex is adjacent to u
                path = [vertex] #We initiate the path
                while u is not None: #Reconstruction of the cycle
                    path.insert(0, u) #So we don't have to reverse it after (saves time)
                    u = predecessor[u] #We go back where we came from
                return ''.join(path) #Since we only want one cycle we can stop the function when and if we find one cycle
    return False

print(bfs(example_graph2, 'A'))

ACA
```

Question 4

a) Proof with BFS: Let $v_1, v_2, \dots, v_{(2n + 1)}$ be a sequence representing an odd cycle. Then we can choose v_1 as the starting point of our cycle by definition. If we then color v_1 as color 1, we have to color v_2 and $v_{(2n + 1)}$ as color 2. We can choose v_2 as our next move of BFS (v_2 goes from the queue to explored and $v_{(2n + 1)}$ stays in the queue.). We then can go to only v_3 , coloring it with color 1. Repeating this, we get all odd indexed vertices colored with color 1 and all even indexed vertices colored with color 2. So we have to color vetrex $v_{(2n + 1)}$ withcolor1, butthatwouldmeanthatadjacentvertices v_1 and $v_{(2n + 1)}$ would be colored with the same color. Proof follows. Note: We could also prove this by induction.

b) Let's assume that G has no odd-lenght cycles but a valid coloring doesn't exist. That means, that there is a pair of adjacent vertices with the same color. For our starting point of BFS we can use an arbitrary vertex, naming it v_1 . We then mark the two adjacent vertices as v_2 and v_{2k} , since the cycle has to be even. Using the BFS algorithm, we can move along the cycle until we reach one of the adjacent same-colored vertices. If the color of the starting vertex and the adjacent same-colored vertices is the same, then the index of the first discovered same-colored vertices has to be odd (proved in a)). If we then start from v_1 and continue to v_{2k} , coloring it the other color, we can eventually reach the second vertex of the two adjacent same-colored vertices. We have assumed that the color of it is the same as that of v_1 , which means that the index of it has to be odd. Contradiction. If we assume that the coloring of the two adjacent same-colored vertices is different from the coloring of v_1 , we reach the same conclusion, which is a contradiction with the even-ness of the cycle.

Question 5

An example of the graph G would have $V = \{s, a, b, c, d\}$ and $E = \{s \rightarrow a, s \rightarrow b, a \rightarrow c, a \rightarrow d, b \rightarrow c, b \rightarrow d\}$. A good set of tree edges E_π would be $\{s \rightarrow a, s \rightarrow b, a \rightarrow c, b \rightarrow d\}$. This way, for each vertex the unique path from s to that vertex is the shortest path. The set of edges E_π cannot be produced by BFS, since when the algorithm reaches either vertex a or b, it would see both c and d as adjacent vertices, making a (or b) the predecessor of both c and d, which is not the case in the graph (V, E_π) .